```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import requests

from io import BytesIO
from scipy.stats import chi2_contingency, ttest_ind
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    ConfusionMatrixDisplay
)
from IPython.display import display, Markdown

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

```
url = (
    "https://raw.githubusercontent.com/"
    "IBM/telco-customer-churn-on-icp4d/"
    "master/data/Telco-Customer-Churn.csv"
)

response = requests.get(url, timeout=60)
response.raise_for_status()

df = pd.read_csv(BytesIO(response.content))

print("Dataset downloaded successfully.")
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset downloaded successfully.
Dataset shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneServi
0	7590-VHVEG	Female	0	Yes	No	1	N
1	5575-GNVDE	Male	0	No	No	34	Y
2	3668-QPYBK	Male	0	No	No	2	Y
3	7795-CFOCW	Male	0	No	No	45	N
4	9237-HQITU	Female	0	No	No	2	Y

```
print("Dataset dimensions:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types and non-null counts:")
df.info()
```

```
Dataset dimensions: (7043, 21)

Column names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',

Data types and non-null counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
```

```
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
print("Missing values by column:")
display(df.isnull().sum().to_frame("Missing Values"))

print("\nDuplicate rows:", df.duplicated().sum())

if "customerID" in df.columns:
    print("Duplicate customer IDs:", df["customerID"].duplicated().sum())
```

```
Missing values by column:
```

	Missing Values
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

```
Duplicate rows: 0
Duplicate customer IDs: 0
```

```
df = df.copy()

# Remove whitespace around column names and string values.
df.columns = df.columns.str.strip()

for col in df.select_dtypes(include=["object"]).columns:
    df[col] = df[col].str.strip()

# Convert TotalCharges from text to numeric.
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)

# Convert target to binary form.
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})

# Remove duplicate rows.
df = df.drop_duplicates()

# Remove rows without a valid target.
df = df.dropna(subset=["Churn"])

# Fill blank TotalCharges with zero for zero-tenure customers.
df.loc[
    df["TotalCharges"].isna() & (df["tenure"] == 0),
    "TotalCharges"
] = 0

print("Preprocessing completed.")
print("Cleaned dataset shape:", df.shape)
print("Missing TotalCharges:", df["TotalCharges"].isna().sum())
print("Missing target values:", df["Churn"].isna().sum())
```

```
Preprocessing completed.
Cleaned dataset shape: (7043, 21)
Missing TotalCharges: 0
Missing target values: 0
```

```
print("Numerical summary:")
display(df.describe().T)

print("\nChurn distribution:")
display(
    df["Churn"]
    .map({0: "No Churn", 1: "Churn"})
    .value_counts()
    .to_frame("Customers")
)

print("\nChurn percentage:")
print((df["Churn"].value_counts(normalize=True) * 100).round(2))
```

```
Numerical summary:
```

	count	mean	std	min	25%	50%	75
SeniorCitizen	7043.0	0.162147	0.368612	0.00	0.00	0.00	0.0
tenure	7043.0	32.371149	24.559481	0.00	9.00	29.00	55.0
MonthlyCharges	7043.0	64.761692	30.090047	18.25	35.50	70.35	89.8
TotalCharges	7043.0	2279.734304	2266.794470	0.00	398.55	1394.55	3786.6
Churn	7043.0	0.265370	0.441561	0.00	0.00	0.00	1.0

```
Churn distribution:
```

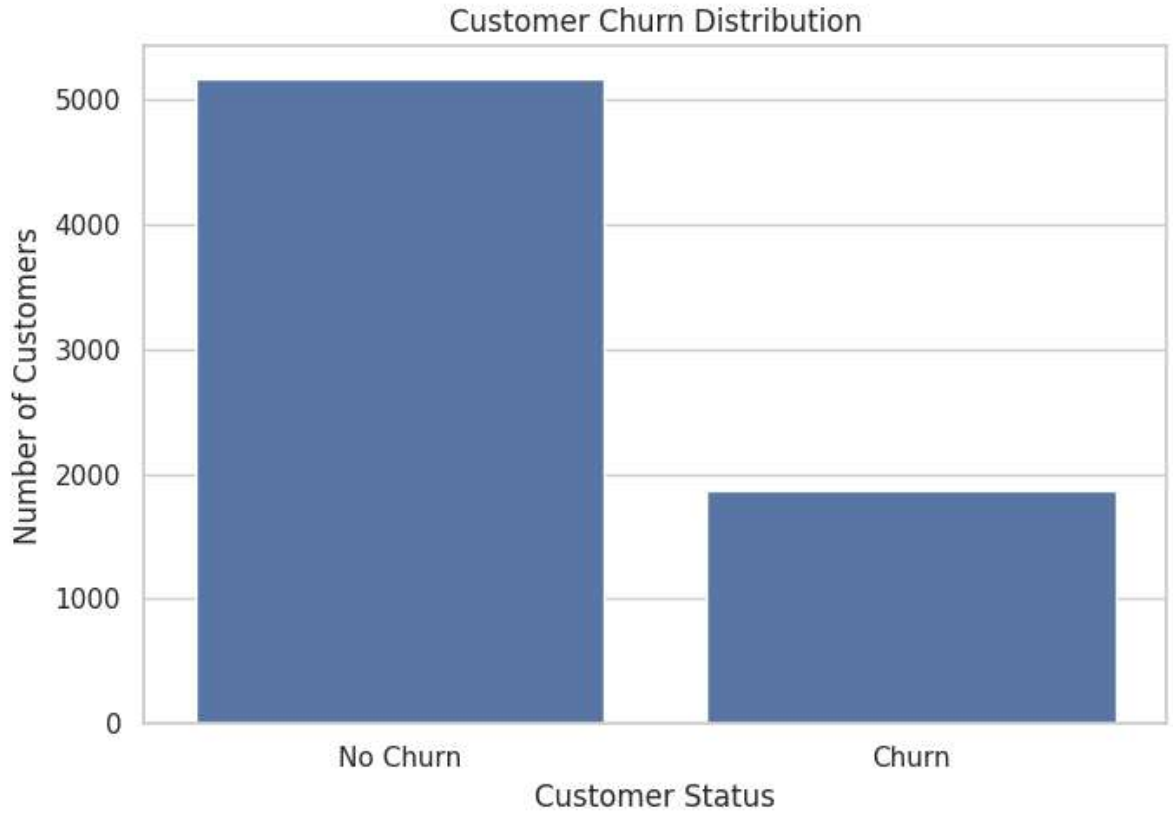
Churn	Customers
No Churn	5174
Churn	1869

```
Churn percentage:
Churn
0    73.46
1    26.54
Name: proportion, dtype: float64
```

```
churn_counts = df["Churn"].map({
    0: "No Churn",
    1: "Churn"
}).value_counts()

plt.figure(figsize=(7, 5))
sns.barplot(
    x=churn_counts.index,
    y=churn_counts.values
)
plt.title("Customer Churn Distribution")
plt.xlabel("Customer Status")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()

display(churn_counts.to_frame("Customer Count"))
```



Customer Count

Churn	
No Churn	5174
Churn	1869

```
contract_churn = pd.crosstab(
    df["Contract"],
    df["Churn"].map({0: "No Churn", 1: "Churn"})
)

display(contract_churn)

contract_rate = (
    df.groupby("Contract")["Churn"]
    .mean()
    .mul(100)
    .sort_values(ascending=False)
)

print("\nChurn rate by contract (%):")
display(contract_rate.to_frame("Churn Rate (%)"))

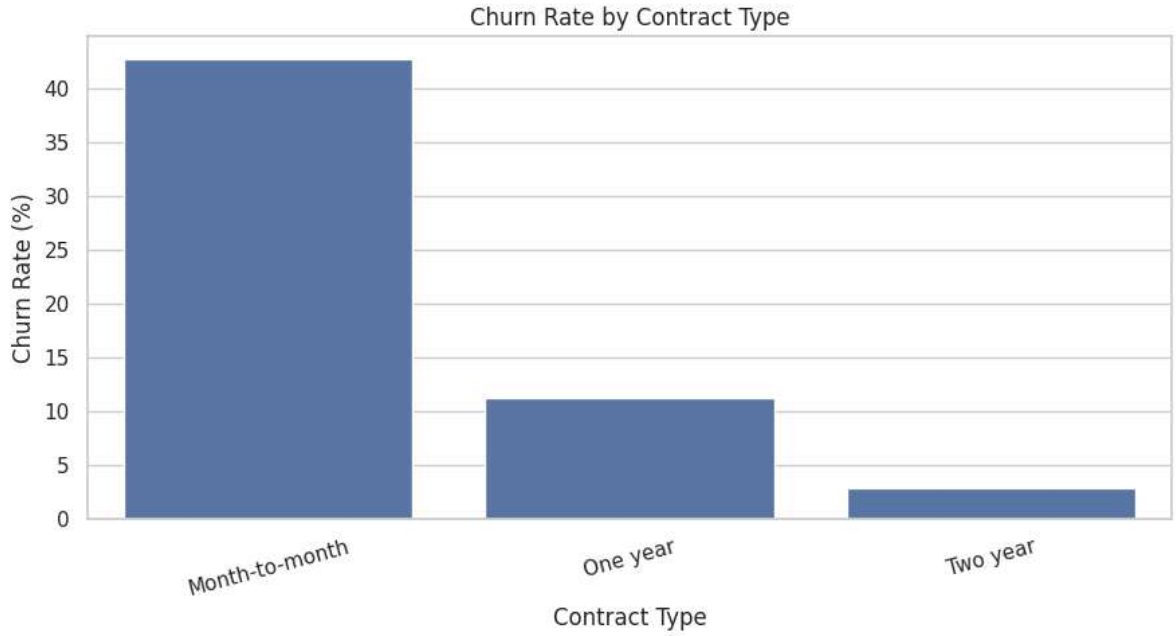
plt.figure(figsize=(9, 5))
sns.barplot(
    x=contract_rate.index,
    y=contract_rate.values
)
plt.title("Churn Rate by Contract Type")
```

```
plt.xlabel("Contract Type")
plt.ylabel("Churn Rate (%)")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

Contract	Churn	Churn	No Churn
Month-to-month	1655	2220	
One year	166	1307	
Two year	48	1647	

Churn rate by contract (%):

Contract	Churn Rate (%)
Month-to-month	42.709677
One year	11.269518
Two year	2.831858



Next steps: Generate code with `contract_churn` New interactive sheet

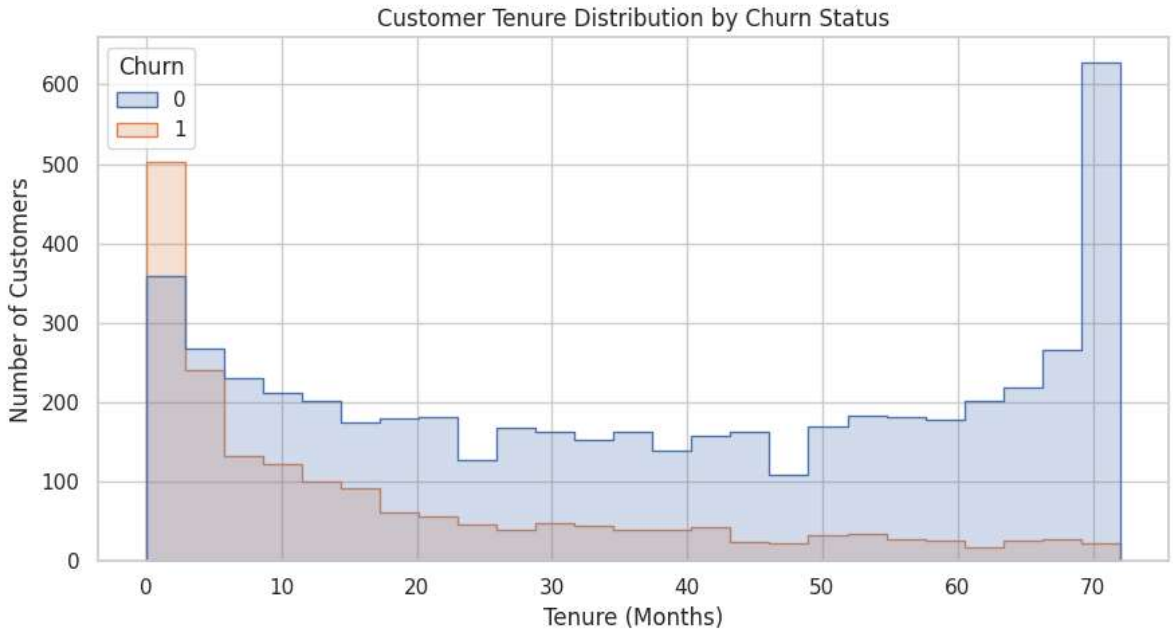
```
plt.figure(figsize=(9, 5))

sns.histplot(
    data=df,
    x="tenure",
    hue="Churn",
    bins=25,
    multiple="layer",
```

```
    element="step"
)

plt.title("Customer Tenure Distribution by Churn Status")
plt.xlabel("Tenure (Months)")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()

display(
    df.groupby("Churn")["tenure"]
    .agg(["count", "mean", "median", "std"])
    .rename(index={0: "No Churn", 1: "Churn"})
)
```



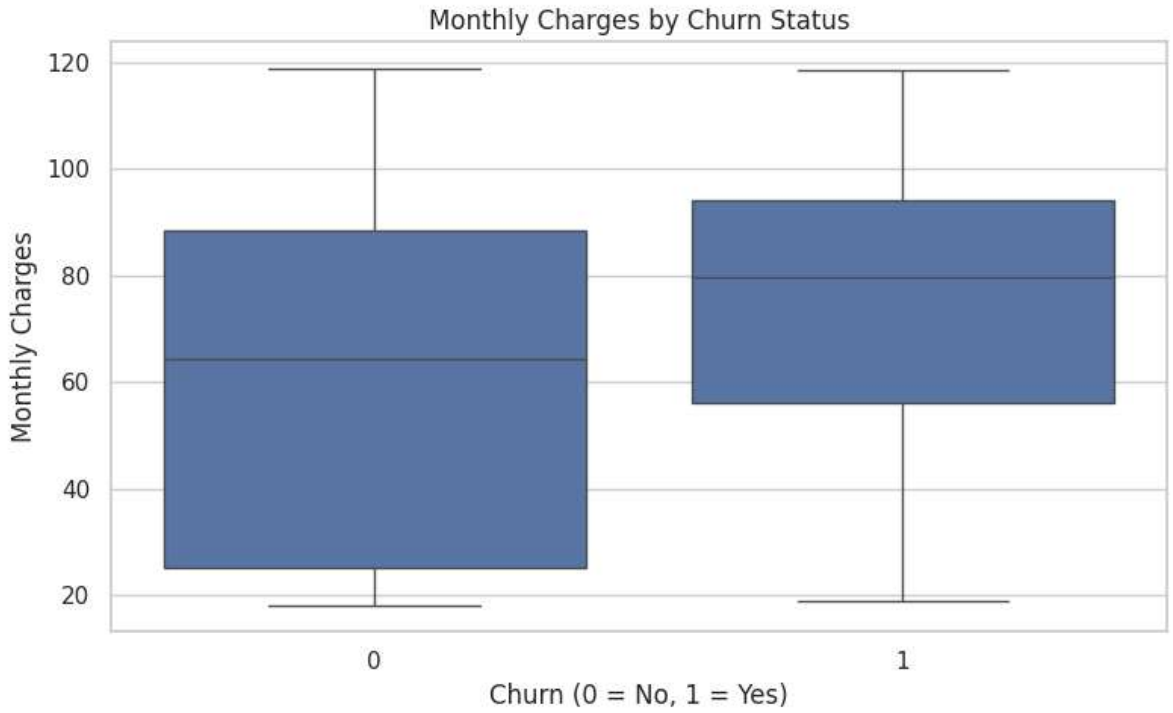
Churn	count	mean	median	std
No Churn	5174	37.569965	38.0	24.113777
Churn	1869	17.979133	10.0	19.531123

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="Churn",
    y="MonthlyCharges"
)

plt.title("Monthly Charges by Churn Status")
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```

```
display(
    df.groupby("Churn")["MonthlyCharges"]
    .agg(["mean", "median", "std"])
    .rename(index={0: "No Churn", 1: "Churn"})
)
```



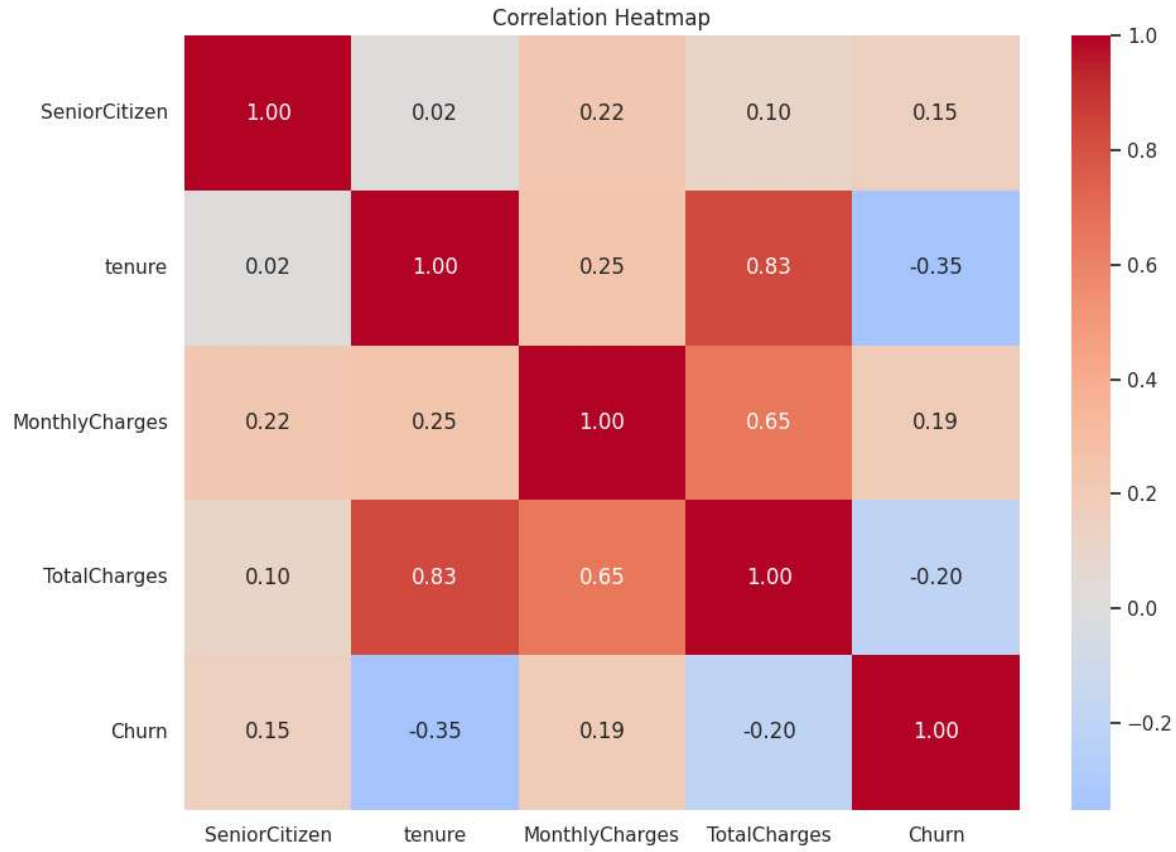
Churn	mean	median	std
No Churn	61.265124	64.425	31.092648
Churn	74.441332	79.650	24.666053

```
numeric_df = df.select_dtypes(include=np.number)

correlation = numeric_df.corr()

plt.figure(figsize=(10, 7))
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```




```
contingency_table = pd.crosstab(
    df["Contract"],
    df["Churn"]
)

chi2, p_value, dof, expected = chi2_contingency(
    contingency_table
)

print(f"Chi-square statistic: {chi2:.4f}")
print(f"Degrees of freedom: {dof}")
print(f"P-value: {p_value:.8f}")

if p_value < 0.05:
    print("Decision: Reject H0.")
    print("Contract type and churn are statistically associated.")
else:
    print("Decision: Fail to reject H0.")
    print("Insufficient evidence of an association.")
```

```
Chi-square statistic: 1184.5966
Degrees of freedom: 2
P-value: 0.00000000
Decision: Reject H0.
Contract type and churn are statistically associated.
```

```
tenure_no_churn = df.loc[df["Churn"] == 0, "tenure"]
tenure_churn = df.loc[df["Churn"] == 1, "tenure"]
```

```
t_stat, p_value = ttest_ind(
    tenure_no_churn,
    tenure_churn,
    equal_var=False
)

print(f"Mean tenure (no churn): {tenure_no_churn.mean():.2f} months")
print(f"Mean tenure (churn): {tenure_churn.mean():.2f} months")
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.8f}")

if p_value < 0.05:
    print("The mean tenure difference is statistically significant.")
else:
    print("The mean tenure difference is not statistically significant.")
```

```
Mean tenure (no churn): 37.57 months
Mean tenure (churn): 17.98 months
T-statistic: 34.8238
P-value: 0.00000000
The mean tenure difference is statistically significant.
```

```
# CustomerID is an identifier, not a useful predictor.
X = df.drop(columns=["Churn", "customerID"], errors="ignore")
y = df["Churn"].astype(int)

# Identify columns by their data types.
numeric_features = X.select_dtypes(
    include=["number"]
).columns.tolist()

categorical_features = X.select_dtypes(
    exclude=["number"]
).columns.tolist()

print("Number of features:", X.shape[1])
print("Numeric features:", len(numeric_features))
print("Categorical features:", len(categorical_features))
print("Target classes:", sorted(y.unique()))
```

```
Number of features: 19
Numeric features: 4
Categorical features: 15
Target classes: [np.int64(0), np.int64(1)]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training records:", len(X_train))
```

```
print("Testing records:", len(X_test))

print("\nTraining churn rate:")
print(round(y_train.mean() * 100, 2), "%")

print("Testing churn rate:")
print(round(y_test.mean() * 100, 2), "%")
```

```
Training records: 5634
Testing records: 1409

Training churn rate:
26.54 %
Testing churn rate:
26.54 %
```

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=2000))
])

print("Preprocessing and classification pipeline created.")
```

```
Preprocessing and classification pipeline created.
```

```
model.fit(X_train, y_train)

print("Logistic Regression model trained successfully.")
```

```
Logistic Regression model trained successfully.
```

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
```

```
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_prob)

print("MODEL PERFORMANCE")
print("-" * 40)
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-score : {f1:.4f}")
print(f"ROC-AUC  : {roc_auc:.4f}")

print("\nClassification report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=["No Churn", "Churn"],
    zero_division=0
))
```

```
MODEL PERFORMANCE
----------------------------------------
Accuracy : 0.8055
Precision: 0.6572
Recall   : 0.5588
F1-score : 0.6040
ROC-AUC  : 0.8421

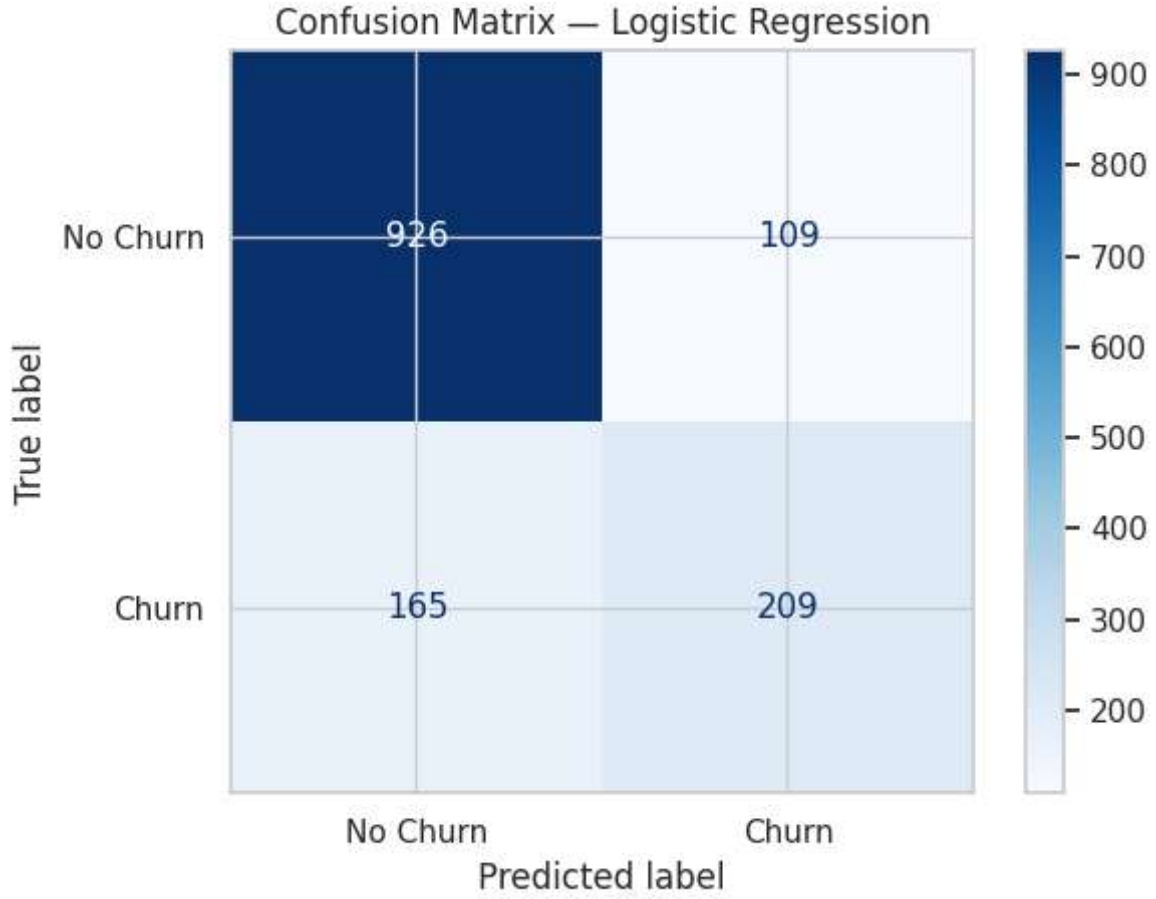
Classification report:
              precision    recall  f1-score   support

    No Churn       0.85      0.89      0.87      1035
       Churn       0.66      0.56      0.60       374

    accuracy                           0.81      1409
   macro avg       0.75      0.73      0.74      1409
weighted avg       0.80      0.81      0.80      1409
```

```
ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    display_labels=["No Churn", "Churn"],
    cmap="Blues"
)

plt.title("Confusion Matrix – Logistic Regression")
plt.tight_layout()
plt.show()
```




```
feature_names = model.named_steps[
    "preprocessor"
].get_feature_names_out()

coefficients = model.named_steps[
    "classifier"
].coef_[0]

importance = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients,
    "Absolute_Importance": np.abs(coefficients)
}).sort_values("Absolute_Importance", ascending=False)

print("Top 15 predictive features:")
display(importance.head(15))

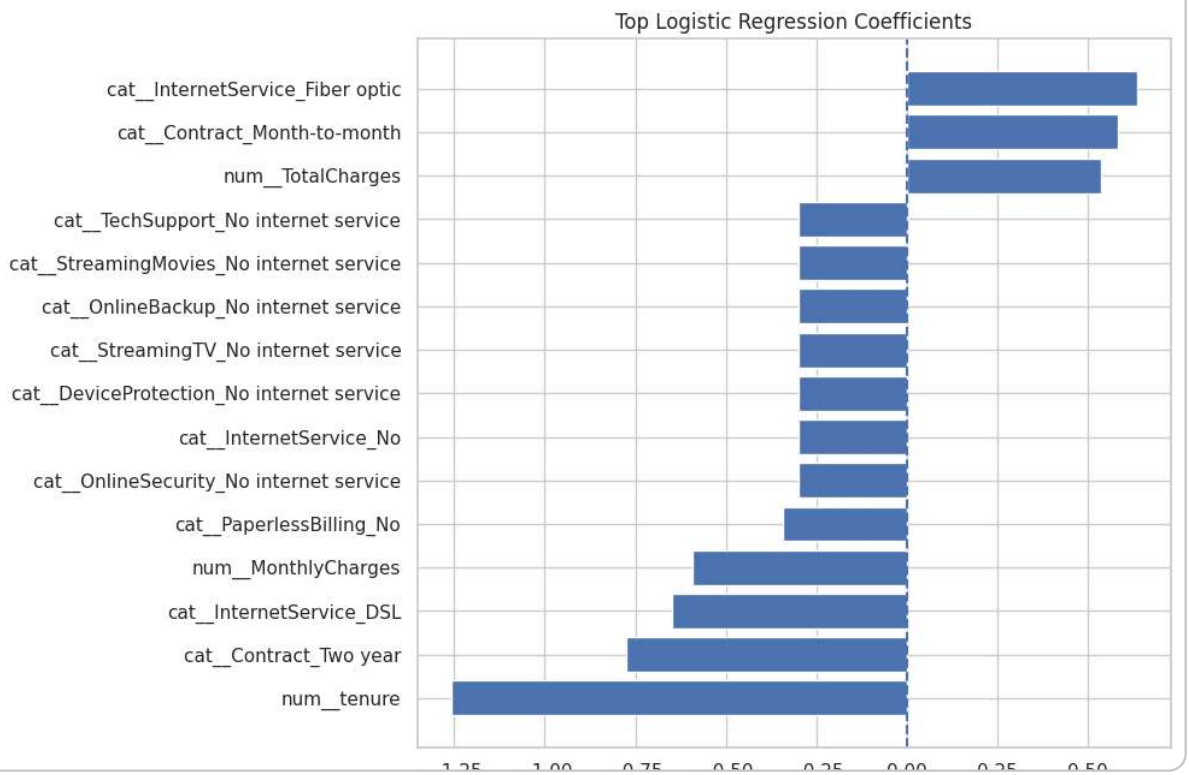
top_features = importance.head(15).sort_values("Coefficient")

plt.figure(figsize=(10, 7))
plt.barh(
    top_features["Feature"],
    top_features["Coefficient"]
)
plt.axvline(0, linestyle="--")
plt.title("Top Logistic Regression Coefficients")
plt.xlabel("Coefficient (positive indicates higher predicted churn odds)")
```

```
plt.tight_layout()
plt.show()
```

```
Top 15 predictive features:
```

	Feature	Coefficient	Absolute_Importance
1	num__tenure	-1.257539	1.257539
38	cat__Contract_Two year	-0.776594	0.776594
15	cat__InternetService_DSL	-0.648646	0.648646
16	cat__InternetService_Fiber optic	0.634195	0.634195
2	num__MonthlyCharges	-0.591863	0.591863
36	cat__Contract_Month-to-month	0.582883	0.582883
3	num__TotalCharges	0.536253	0.536253
39	cat__PaperlessBilling_No	-0.343227	0.343227
25	cat__DeviceProtection_No internet service	-0.300104	0.300104
17	cat__InternetService_No	-0.300104	0.300104
19	cat__OnlineSecurity_No internet service	-0.300104	0.300104
28	cat__TechSupport_No internet service	-0.300104	0.300104
31	cat__StreamingTV_No internet service	-0.300104	0.300104
22	cat__OnlineBackup_No internet service	-0.300104	0.300104
34	cat__StreamingMovies_No internet service	-0.300104	0.300104



```
churn_rate = df["Churn"].mean() * 100

highest_churn_contract = (
    df.groupby("Contract")["Churn"].mean().idxmax()
)

highest_churn_rate = (
    df.groupby("Contract")["Churn"].mean().max() * 100
)

avg_tenure_churn = df.loc[
    df["Churn"] == 1, "tenure"
].mean()

avg_tenure_retained = df.loc[
    df["Churn"] == 0, "tenure"
].mean()

print("BUSINESS INSIGHTS AND RECOMMENDATIONS")
print("=" * 50)

print(f"Overall customer churn rate: {churn_rate:.2f}%")

print(
    f"\nContract type with highest observed churn rate: "
    f"{highest_churn_contract} ({highest_churn_rate:.2f}%)"
)

print(
    f"\nAverage tenure of churned customers: "
    f"{avg_tenure_churn:.2f} months"
)

print(
    f"Average tenure of retained customers: "
    f"{avg_tenure_retained:.2f} months"
)

print("\nRecommended actions:")
print("1. Prioritize retention outreach for customers with high predicted ch
print("2. Investigate contract types with higher observed churn rates.")
print("3. Review service quality, billing issues, and customer support.")
print("4. Evaluate retention offers using controlled experiments.")
print("5. Monitor model recall, precision, and churn rate over time.")
```

```
BUSINESS INSIGHTS AND RECOMMENDATIONS
==================================================
Overall customer churn rate: 26.54%

Contract type with highest observed churn rate: Month-to-month (42.71%)

Average tenure of churned customers: 17.98 months
Average tenure of retained customers: 37.57 months

Recommended actions:
```

1. Prioritize retention outreach for customers with high predicted churn risk
2. Investigate contract types with higher observed churn rates.
3. Review service quality, billing issues, and customer support.
4. Evaluate retention offers using controlled experiments.
5. Monitor model recall, precision, and churn rate over time.
